



מספר התלמיד הנבחן
רשום את כל תשע הספרות

הדבק כאן את
מדבקת הנבחן

האוניברסיטה
הפתוחה



כ"ח בסיון תשע"ט

מס' שאלון - 517

1

ביולי 2019

סמסטר 2019 ב

מס' מועד 84

20905 / 4

שאלון בחינת גמר
20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה 10 עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

שימו לב:

את תשובותיכם רשמו אך ורק במחברת הבחינה.

כתבו בכתב יד ברור ומסודר.

חומר עזר:

ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים

להחזיר את השאלון לאוניברסיטה הפתוחה



שאלה 1 (25 נקודות)

הרחיבו את שפת LET (שפת "ויהי") עם ביטוי חדש בשם max.

להלן הדקדוק המגדיר את התחביר החדש עבור ביטוי max :

Expression ::= max (identifier)

max-exp (var)

כאשר,

var הוא שם של משתנה שהוגדר כבר (יכול להיות אף מספר פעמים עד כה).

כזכור לכם, כאשר נדרש לחשב את ערכו של משתנה תחת סביבה נתונה, מוחזר ערכו של המופע האחרון של המשתנה בסביבה.

בשאלה זו, נדרש שביטוי max-exp יחזיר את ערכו המקסימלי של המשתנה הנתון ע"י var מבין כל ערכיו הקיימים בסביבה הנתונה. יש להתייחס רק לערכי var שהם מספרים בלבד. במידה ולא קיים כלל ערך מספרי ל var, ביטוי max-exp יחזיר שגיאה במקרה זה (ע"י eopl:error).

להלן דוגמא להמחשת השימוש בביטוי החדש :

```
let m = 4
in
  let m = zero?(m)
  in
    let m= 7
    in
      let m= 5
      in
        -(max(m), m)
```

בדוגמה זו, נתון ביטוי let מקונון המגדיר מספר מופעים של m (4 מופעים שונים, 3 מהם מספריים, ואחד הוא בוליאני) ערכו של כל הביטוי הוא כערכו של ה-body של ביטוי ה-let האחרון, שהוא -(max(m),m). ביטוי זה הוא פעולת חיסור בין, תוצאת הביטוי max (m) לבין תוצאת הביטוי m. חישוב הביטוי max(m) אמור לסקור את כל ערכי m הקיימים בסביבה איתה מחושב ביטוי זה ומתוכם לברור את אלה שהם מספריים (4, 7 ו-5) ולהחזיר את ערכו של המקסימלי מבניהם, שהוא 7. ערכו של הביטוי m הוא כערכו האחרון של m בסביבה שהוא 5. לכן במקרה זה, תוחזר תשובה (num-val 2) שהיא תוצאת החיסור בין 7 ל-5.

(המשך השאלה בעמוד הבא)

א) (7 נקודות)

להלן נתונים ה-scanner, וה-parser של שפת "ויהי". כתבו מהו השינוי שיש להכניס בהגדרות שפת LET כך שיתאים להגדרת הדקדוק החדש. **את תשובותיכם כתבו אך ורק במחברת הבחינה.**

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% " (arbno (not #\newline)))) skip)
    (identifier (letter (arbno (or letter digit "_" "-" "?")))) symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
    (expression (number) const-exp)
    (expression ("-" "(" expression "," expression ")") diff-exp)
    (expression ("zero?" "(" expression ")") zero?-exp)
    (expression ("if" expression "then" expression "else" expression) if-exp)
    (expression (identifier) var-exp)
    (expression ("let" identifier "=" expression "in" expression) let-exp)
  ))
```

(המשך השאלה בעמוד הבא)

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת "ויהי" כפי שמופיעה בספר הלימוד. יש לעדכנה כך שתחשב את ביטוי *max-exp* על פי הדקדוק החדש. את התשובה כיתבו במחברת הבחינה.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "ויהי".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

הפרוצדורה *value-of* (כפי שמופיעה בספר בעמוד 71):

;;value-of : Exp x Env ----> ExpVal

```
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))
      (var-exp (var) (apply-env env var))
      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val (- num1 num2))))))
      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1) (bool-val #t) (bool-val #f)))))
      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1) (value-of exp2 env) (value-of exp3 env))))
      (let-exp (var exp1 body)
        (let ((val1 (value-of exp1 env)))
          (value-of body (extend-env var val1 env))))
    )))
```

שאלה 2 (25 נקודות)

להלן תוכנית בשפת PROC (שפת "וישגור") :

```
let x=5
in
  let x= proc (x) (x 10)
  in
    (x proc (x) -(x,7))
```

עקבו אחר התוכנית באמצעות תרשימי סביבות. וענו היטב ובצורה ברורה על השאלות הבאות :

א) (12 נקודות)

מהו פלט התוכנית? (הסבירו במילים ובאמצעות תרשימי הסביבות שהכנתם)

ב) (13 נקודות)

מהו ערכו של כל x המוזכר בביטוי? (ציינו זאת בתרשימים השונים או הסבירו במילים תוך תיאור הסביבה בה נשמר כל x)

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם `apply`.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression ::= apply identifier in identifier during Expression

`apply-exp (id1 id2 exp )`

לביטוי `apply-exp` המרכיבים הבאים:

- **id1** שם של משתנה שבו תשתמש הפרוצדורה המוגדרת ע"י `id2`
- **id2** שם של פרוצדורה שהוגדרה קודם לכן. במסגרת חישוב הביטוי `exp`, אם יש דרישה להפעיל את הפרוצדורה `id2` היא תופעל עם ארגומנט ששמו מצויין ע"י `id1` במקום שם הארגומנט שהיה נתון בזמן הגדרתה.
- **exp** – ביטוי שיש לחשב ולהחזיר את תוצאתו. הביטוי יחושב כאשר שמו של ארגומנט הפרוצדורה יהיה באופן זמני כשם המצוין ע"י `id1`. לאחר חישוב הביטוי `exp` הפרוצדורה תחזור להגדרתה המקורית ותשתמש בשם הארגומנט איתו הוגדרה

להלן דוגמא:

```
let z= 10
in
  let y= 30
  in
    let p= proc (x) -(x,y)
    in
      let q= apply y in p during (p 100)
      in
        apply z in p during -(q, (p 300))
```

בדוגמה זו, מוגדר משתנה `z` שערכו 10. ומשתנה `y` שערכו 30. וכן פרוצדורה המצפה לפרמטר בשם `x` ומחזירה את ערכו של הביטוי `-(x,y)`

לאחר מכן, מוגדר משתנה `q` המקבל את ערכו של ביטוי `apply`.

ביטוי `apply` יפעיל את הפרוצדורה `p` עם פרמטר שערכו 100 אך שם הארגומנט של `p` לא יהיה `x` אלא `y`, ולכן במסגרת חישוב הפרוצדורה לא יוכר `y` שערכו 30 אלא `y` המשמש כארגומנט זמני של הפרוצדורה עם ערך 100. ולכן ערכו של `q` יהיה 0.

לאחר מכן מוחזר ערכו של הביטוי `apply` השני, שבו, שוב מופעלת הפרוצדורה `p` הפעם עם ארגומנט ששמו ייקרא `z` והוא יקבל ערך 300. הפרוצדורה במקרה תחזיר 270 מאחר ובהפעלה זו, `y` המוזכר בגוף הפרוצדורה מוכר עם ערך 30 כפי שהיה בזמן ההגדרה.

ולכן בסה"כ מוחזרת תשובה של `-270 = 0-270`

(המשך השאלה בעמוד הבא)

א. (7 נקודות)

להלן נתונים ה-scanner וה-parser של שפת "וירמוז". עליכם להשלים את החסר בריבוע המסומן. את התשובה כיתבו במחברת הבחינה.

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% (arbno (not #\newline))) skip)
    (identifier(letter (arbno (or letter digit "_" "-" "?")))
      symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
    (expression (number) const-exp)
    (expression ("-" "(" expression "," expression ")")
      diff-exp)
    (expression ("zero?" "(" expression ")") zero?-exp)
    (expression
      ("if" expression "then" expression "else" expression)
      if-exp)
    (expression (identifier) var-exp)
    (expression ("let" identifier "=" expression "in" expression)
      let-exp)
    (expression ("proc" "(" identifier ")" expression)
      proc-exp)
    (expression "(" expression expression ")" call-exp)
    (expression
      ("letrec"
        (arbno identifier "(" identifier ")" "=" expression)
        "in" expression)
      letrec-exp)
    (expression ("set" identifier "=" expression)
      assign-exp)
    (expression (
      )
      apply-exp)))
```

את התשובה כיתבו במחברת הבחינה

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו- *apply-procedure* של שפת "וירמוז".
הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.
את התשובה כיתבו במחברת הבחינה.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))

      (var-exp (var) (deref (apply-env env var)))

      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val
             (- num1 num2))))))

      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1)
                (bool-val #t)
                (bool-val #f))))))

      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

      (let-exp (var exp1 body)
        (let ((v1 (value-of exp1 env)))
          (value-of body
                     (extend-env var (newref v1) env))))

      (proc-exp (var body)
        (proc-val (procedure var body env)))

      (call-exp (rator rand)
        (let ((proc (expval->proc (value-of rator env)))
```



```

        (arg (value-of rand env)))
      (apply-procedure proc arg)))

(letrec-exp (p-names b-vars p-bodies letrec-body)
  (value-of letrec-body
    (extend-env-rec* p-names b-vars p-bodies env)))

(assign-exp (var exp1)
  (begin
    (setref!
      (apply-env env var)
      (value-of exp1 env))
    (num-val 27)))))

(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body
          (extend-env var (newref val) saved-env))))))

```

המשך הבחינה בעמוד הבא

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let m=5
in
  let p = proc (x:?) (x 4)
  in
    let q= (p proc (y:?) zero? (-(y,7)))
    in
      if q then -(m 7) else m
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב-substitution ו-unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !